SOFTWARE ENGINEERING ACADEMIC PROJECT — MID-SEMESTER EVALUATION

Department of Computer Science & Engineering

Software Requirements Specification

Project: AutoTestHub (Web Automation & Testing Platform)

IEEE 830 STANDARD COMPLIANT • VERSION 1.0

PROJECT & COURSE METADATA

System Name: AutoTestHub

Document ID: ATH-SRS-2026-V1

Course: Software Engineering (CS302)

Academic Section: Section [X] • Group [XX]

Evaluation Stage: Mid-Semester Review (40–60%)

PROJECT DEVELOPMENT TEAM

Shristi Neekhra (Integration & Execution Service)

Shifa Siddiqui (Backend Architecture & Database)

Shruti Gupta (Selenium Engine & Worker Module)

Shriya Agarwal (Frontend Control Desk & Dashboard)

Instructor / Guide: [Supervisor Name]

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) details the functional, non-functional, and interface requirements for **AutoTestHub**, an automated web testing and orchestration platform. This baseline document governs the mid-semester implementation milestone (representing 40–60% of total system scope) and defines the technical contracts required for test definition ingestion, cross-tier IPC execution, dynamic browser action dispatching, and persistent test auditing.

1.2 Scope of the Product

AutoTestHub bridges the complexity gap between raw, fragile Selenium automation scripts and modern web development workflows. The software delivers a modular, multi-tier system enabling developers and QA engineers to define declarative web tests in JSON, execute them in real Google Chrome instances (both headless and windowed), capture timestamped screenshot artifacts, evaluate title assertions, and persist structured audit logs in MongoDB via REST APIs.

1.3 Document Conventions

This document follows IEEE Std 830-1998 guidelines. Requirements are uniquely identified using prefixes:

- `FR-xx`: Functional Requirements governing system behavior and operational logic.
- `NFR-xx`: Non-Functional Requirements specifying performance, reliability, security, and usability benchmarks.
- `INT-xx`: External Interface Requirements specifying REST and inter-process communication protocols.

2. Overall Description

2.1 Product Perspective

AutoTestHub operates as a four-tier architecture consisting of:

1. **Client Tier:** The interactive Selenium Control Desk providing execution controls, simulation preview, and real-time terminal output streaming.
2. **API Gateway Tier:** Node.js and Express REST services exposing endpoints for test creation, test retrieval, and execution triggering.
3. **Execution Bridge Tier:** An asynchronous Node.js execution service managing subprocess spawning, JSON serialization via stdin, and regex-based stdout extraction.
4. **Automation Worker Tier:** A Python 3.12 Selenium WebDriver execution engine utilizing Selenium Manager, explicit WebDriverWait constructs, and automatic screenshot generation.

2.2 User Classes & Characteristics

User Class	Technical Profile	System Interactions
QA Automation Engineer	Proficient in testing methodologies, locators (CSS/XPath), and assertion criteria.	Authors JSON test definitions, triggers test suites, inspects screenshot evidence, and reviews regression logs.
Software Developer	Full-stack or backend engineer maintaining REST services and web interfaces.	Executes local sanity tests via REST endpoints, integrates test scripts into local branches, and validates PRs.
Engineering Lead / Evaluator	Academic supervisor or project lead monitoring velocity and test pass rates.	Inspects the Control Desk dashboard, reviews execution history tables, and evaluates compliance with specifications.

3. Functional Requirements (FR)

The following functional requirements specify the operational capabilities implemented for the mid-semester evaluation:

Req ID	Feature Name	Requirement Description & Verification Rule	Priority
`FR-01`	JSON Test Definition Ingestion	The system shall ingest structured test cases defined in JSON format containing `testCaseId`, `name`, target `url`, and an ordered list of `actions`. Incomplete schemas must be rejected with diagnostic validation errors.	MUST
`FR-02`	URL Safety Validation	The worker engine must parse and validate all navigation URLs, enforcing `http://` or `https://` schemes. Attempts to execute `file://`, `javascript:`, or arbitrary shell strings must be rejected with a `ValueError` before browser launch.	MUST
`FR-03`	Dynamic Locator Resolution	The action dispatcher shall dynamically resolve locator strategies across four standard types: `id`, `name`, `css`, and `xpath`. Unsupported locator strategies must fail fast prior to execution.	MUST
`FR-04`	Explicit Wait Synchronization	All interactive element actions (`TYPE`, `CLICK`) must employ explicit synchronization using `WebDriverWait` with `visibility_of_element_located` and `element_to_be_clickable` rather than arbitrary sleep pauses.	MUST
`FR-05`	Text Input Action (`TYPE`)	The system shall wait for element visibility, clear existing values from the targeted input field, and enter the specified string value with timeout protection (default: 10.0 seconds).	MUST
`FR-06`	Element Click Action (`CLICK`)	The system shall wait for element clickability and perform a native WebDriver mouse click on the targeted DOM element with timeout protection (default: 10.0 seconds).	MUST
`FR-07`	Title Assertion (`ASSERT_TITLE`)	The engine shall explicitly wait up to 5.0 seconds for the browser page title to match the expected string. In case of mismatch, the engine must record an assertion failure and raise an informative error.	MUST
`FR-08`	Automated Screenshot Capture	The system must automatically capture a full viewport PNG screenshot upon test completion—both on successful test pass and on runtime failure/timeout. Screenshots must be stored with unique timestamped hash filenames.	MUST
`FR-09`	Subprocess Lifecycle Safety	The worker engine must wrap all WebDriver operations in strict `try-finally` blocks ensuring `driver.quit()` is guaranteed to execute, preventing zombie Chrome background processes.	MUST
`FR-10`	Strict JSON Output Contract	The Python runner shall output exactly ONE single-line JSON string to standard output (`stdout`) representing the execution summary, while redirecting all logging messages to `stderr`.	MUST
`FR-11`	Express REST API Gateway	The Node.js backend shall expose RESTful endpoints: `POST /api/tests` (create test case), `GET /api/tests` (list test cases), and `POST /api/tests/:id/run` (trigger execution pipeline).	MUST
`FR-12`	MongoDB Result Persistence	The execution service shall asynchronously persist execution outcomes (`testCaseId`, `status`, `executionTime`, `screenshot`, `message`, `errorMessage`) in MongoDB using Mongoose models.	MUST

Mid-Semester Compliance Status: 100% Implemented

> All 12 functional requirements listed above are currently implemented, integrated, and validated across unit and integration test harnesses in the repository snapshot.

4. Non-Functional Requirements (NFR)

Req ID	Category	Specification & Quality Standard
`NFR-01`	Execution Performance	The Python automation engine overhead (excluding target network latency) must not exceed 250 milliseconds per action. Complete sandbox integration tests must complete within 2.5 seconds.
`NFR-02`	Fault Tolerance & Recovery	In the event of network disconnection, missing DOM elements, or Python runtime exceptions, the backend must catch errors, return a structured `FAILED` response, and persist diagnostic logs without hanging the server process.
`NFR-03`	Security & Sandboxing	The system shall reject unsanitized file paths, non-HTTP protocols, and code injection attempts. API keys and database credentials must be managed strictly through `.env` files and excluded from version control.
`NFR-04`	Zero-Config Usability	The automation engine shall utilize Selenium 4's built-in **Selenium Manager** to automatically detect host Chrome versions and resolve compatible ChromeDriver binaries without manual driver installation.
`NFR-05`	Platform Portability	The codebase must run seamlessly across standard operating systems (macOS, Windows 11, Ubuntu Linux) using standard environment variables and cross-platform path resolution (`path.join()` and `os.path`).
`NFR-06`	Database Integrity	All MongoDB records must enforce Mongoose schema validation. The `TestResult` collection must maintain referential integrity with the `TestCase` collection via indexed foreign key ObjectId references.
`NFR-07`	Maintainability	Source code must follow modular separation of concerns. Action handlers, locator resolvers, and validators must be isolated into single-responsibility Python modules.

5. Requirements Prioritization (MoSCoW Matrix)

Category	Scope Items	Semester Timeline
Must Have (M)	JSON test parser, dynamic locators (ID, CSS, Name, XPath), safe URL validation, explicit waits (Type & Click), title assertions, automated screenshot generation, clean process exit, Express API routes, MongoDB schemas.	COMPLETED (MID-SEM)
Should Have (S)	Interactive Selenium Control Desk web UI, real-time terminal log viewer, simulation playback, end-to-end integration test harness with mock HTTP sandbox.	COMPLETED (MID-SEM)
Could Have (C)	Multi-test batch queuing, background job runner (BullMQ/Redis), advanced assertion actions (text contains, element visible, attribute equals), React component dashboard.	IN PROGRESS (POST-MIDSEM)
Won't Have (W)	Distributed cloud browser testing (SauceLabs/BrowserStack), multi-tenant enterprise SSO, automated visual regression pixel-diffing (reserved for future versions).	Deferred to Final Release

6. Traceability Matrix

Req ID	Implemented Component	Target Verification Suite	Mid-Sem Status
`FR-01, FR-02`	`selenium-worker/runner.py` , `actions/open_url.py`	`tests/test_validation.py::test_valid_urls`	VERIFIED
`FR-03,`	`selenium-worker/actions/__init__.py`	`tests/test_validation.py::test_locator_resolution`	VERIFIED

FR-04			
FR-05, FR-06	actions/type_text.py, actions/click.py	tests/test_integration.py::test_type_and_click_interaction	VERIFIED
FR-07, FR-08	validators/result_validator.py, browser.py	tests/test_integration.py::test_matching_title_assertion	VERIFIED
FR-09, FR-10	selenium-worker/runner.py	tests/test_integration.py::test_cleanup_after_failure	VERIFIED
FR-11, FR-12	backend/services/executionService.js	backend/testExecution.js pipeline run	VERIFIED